

TimeLineHelpRequest Service

1. Service Overview

1.1) Service Purpose

จัดเก็บและจัดการคำร้องขอความช่วยเหลือ ในสถานการณ์ภัยพิบัติ โดยมีการบันทึกประวัติการเปลี่ยนแปลงสถานะ ทุกขั้นตอนในรูปแบบ Append-only Timeline และทำหน้าที่รวบรวม ข้อมูลจาก Service อื่นๆ นำมาร้อยเรียงเป็นไทม์ไลน์ที่อ่านง่าย ทำให้ผู้ใช้งานหรือศูนย์ประสานงานสามารถตรวจสอบสถานะล่าสุดของแต่ละคำร้องผ่าน Request ID ได้ อย่างครบถ้วนแบบ Real-time ตั้งแต่เริ่มรับแจ้งเหตุ ไปจนถึงการส่งตัวผู้ป่วย หรือการเบิกจ่ายสิ่งของบริจาค เพื่อให้เกิดความโปร่งใสและสามารถตรวจสอบย้อนหลังได้ บริการนี้ให้ความช่วยเหลือของตนเองแยกจากสถานะของภัยพิบัติหลัก

1.2) Pain point

ข้อมูลสถานะการให้ความช่วยเหลือ เช่น ส่งรพพยาบาล, ขอเสบียงอาหาร มักมีการเปลี่ยนแปลงตลอดเวลา หากใช้ระบบที่อัปเดตแบบเขียนทับ จะทำให้ประวัติข้อมูลเดิมสูญหาย เจ้าหน้าที่ Dispatcher จะไม่สามารถติดตามลำดับเหตุการณ์หาผู้รับผิดชอบ หรือวิเคราะห์สาเหตุของความล่าช้าในการเข้าช่วยเหลือได้

1.3) Target Users

Dispatcher (เจ้าหน้าที่ประสานงาน)

1.4) Service Boundary

- In-scope

สร้างคำร้องขอความช่วยเหลือใหม่และออกรหัส Request ID (เช่น REQ-ABCD1234) เพื่อใช้เป็น Correlation ID ให้กับ Service อื่นๆ

อัปเดตสถานะและรวบรวมประวัติ: จัดการและอัปเดตสถานะของคำร้อง เช่น NEW, sent_request_hospital, hospital_confirm, DISPATCHED, CANCELLED โดยรวบรวมจากการรับแจ้งผ่าน REST API สำหรับ Healthcare และผ่าน Event-Driven Message สำหรับ TrackDonation บันทึก Timeline อัตโนมัติ: สร้าง Update Log พร้อม Timestamp และบันทึกผู้ดำเนินการ (Updated By) อัตโนมัติทุกครั้งที่มีความเคลื่อนไหว เพื่อร้อยเรียงเป็นไทม์ไลน์

ระบบ Upsert อัตโนมัติ: สร้างประวัติใหม่ให้ทันที หากได้รับ Event แจ้งเตือนที่มี Request ID ซึ่งยังไม่มีในระบบ เพื่อป้องกันปัญหาข้อมูลสูญหาย

- Out-of-scope

การตรวจสอบและยืนยันการเกิดภัยพิบัติหลัก, การจัดการข้อมูลรายละเอียดผู้ประสบภัย, การวางแผนเส้นทางเดินรถของทีมกู้ภัย, การอนุมัติเบิกจ่ายและตัดสต็อกสิ่งของบริจาค

1.5) Autonomy / Decision Logic

มีลอจิกในการจัดการข้อมูลที่ยืดหยุ่น เมื่อได้รับ Event อัปเดตจาก Service อื่น (เช่น TrackDonation ส่งผ่าน SNS) ระบบจะค้นหา Request ID ก่อน ถ้าพบจะทำการเพิ่ม Log ต่อท้าย แต่หากไม่พบ ระบบจะตัดสินใจ สร้างประวัติ คำร้องใหม่ ให้อัตโนมัติ โดยไม่ปฏิเสธ ข้อมูลนั้นทิ้ง เพื่อให้ Flow งานของ Service อื่นทำงานต่อได้

Event-Driven State Management: การเปลี่ยนสถานะ (State) ของคำร้อง จะถูกขับเคลื่อนโดย Event ที่ได้รับ มา เช่น หากได้รับ eventType: DONATION_ALLOCATED ระบบจะประเมินและบังคับเปลี่ยนสถานะเป็น DISPATCHED อัตโนมัติ

Automated Timestamp & Traceability: ทุกครั้งที่มีการอัปเดต ไม่ว่าจะมาจากช่องทาง Sync (API) หรือ Async (SNS) ระบบจะมีอิสระในการสร้างชุดข้อมูลอัปเดต (Update Log) เอง โดยจัดเตรียม UUID ของ Log, ประทับเวลา (Timestamp) ปัจจุบัน, และบันทึกแหล่งที่มา (Source/Updated By) ให้เสร็จสรรพ โดยไม่ต้องพึ่งพาข้อมูลเหล่านี้จาก Client ที่ยิงเข้ามา

2. Sync Contract

Base URL: [https://\[API-GATEWAY-URL\]/v1](https://[API-GATEWAY-URL]/v1)

API 2.1: Create Help Request (สร้างคำร้องทั่วไป)

- Method/Path: POST /v1/help-requests
- Description: สร้างคำร้องขอความช่วยเหลือใหม่และตั้งสถานะเริ่มต้นเป็น NEW
- Request Body: incident_id (Required), request_type (Required), reporter_id, description, location_id
- Rules: incident_id และ request_type ต้องไม่เป็นค่าว่าง
- Responses:
 - 201 Created: สร้างสำเร็จ คืนค่า request_id, current_status
 - 400 Bad Request (Error): ส่งฟิลด์ที่ Required มาไม่ครบ
 - 500 Internal Server Error (Error): ระบบเซิร์ฟเวอร์หรือฐานข้อมูลขัดข้อง

API 2.2: Healthcare Integration (รับแจ้งเหตุและอัปเดตจาก Healthcare)

- Method/Path: POST /v1/healthcare
- Description: Endpoint เฉพาะสำหรับ Healthcare Service ในการแจ้งเคสผู้ป่วยใหม่ หรืออัปเดตสถานะการส่งตัว
- Request Body: incident_id (Required), request_id (Required สำหรับอัปเดต), status, description, lat, lon
- Responses:
 - 200 OK: อัปเดตสำเร็จ หรือ สร้างใหม่สำเร็จ
 - 400 Bad Request (Error): ไม่ได้ส่ง incident_id
 - 404 Not Found (Error): ส่ง request_id มาอัปเดต แต่หาไม่เจอในระบบ
 - 500 Internal Server Error (Error): ระบบขัดข้อง

API 2.3: View Timeline History (ดึงข้อมูลประวัติไทม์ไลน์)

- Method/Path: GET /v1/help-requests/{request_id}/timeline
- Description: ดึงข้อมูลประวัติการเปลี่ยนสถานะทั้งหมดของคำร้องเรียงตามลำดับเวลา เพื่อนำไปแสดงผลบน Dashboard
- Responses:
 - 200 OK: คืนค่าข้อมูล Item ทั้งก้อน พร้อม Array ของ timeline_logs
 - 404 Not Found (Error): ไม่พบ request_id นี้ในฐานข้อมูล

API 2.4: Append Status Update (อัปเดตสถานะ)

- Method/Path: PATCH /v1/help-requests/{request_id}/status
- Description: อัปเดตสถานะของคำร้อง โดยระบบจะต่อท้ายประวัติใหม่ลงใน Array Timeline อัตโนมัติ
- Request Body: new_status (Required), note (Optional)
- Responses:
 - 200 OK: อัปเดตสำเร็จ
 - 400 Bad Request (Error): ไม่ได้ส่งค่าสถานะใหม่มา
 - 404 Not Found (Error): ไม่พบ request_id นี้ในระบบ

API 2.5: List Help Requests by Incident (ค้นหาคำร้องตามเหตุการณ์)

- Method/Path: GET /v1/help-requests?incident_id={incident_id}
- Description: ค้นหาและแสดงรายการคำร้องทั้งหมดที่อยู่ภายใต้เหตุการณ์เดียวกัน เพื่อให้ Dispatcher ดูภาพรวม
- Responses:
 - 200 OK: คืนค่า Array ของคำร้องทั้งหมดในเหตุการณ์นั้น
 - 400 Bad Request (Error): ไม่ได้แนบ Query Parameter incident_id

API 2.6: Verify Incident (ตรวจสอบกับ Incident Reporter Service)

- Method/Path: GET /v1/incidents/{incident_id}
- Expectation: คาดหวัง HTTP 200 OK เพื่อยืนยันว่าภัยพิบัติมีอยู่จริง (มี Fallback Timeout 3 วินาที หาก Error หรือ Timeout ระบบจะให้ค่าเป็น incident_verified: false และทำงานต่อได้โดยไม่พัง)

3. Async Contract

3.1 Consumer: รอรับ Event การเบิกจ่ายสิ่งของ (Subscribe)

- Trigger Mechanism: AWS SNS รอรับข้อมูลจาก TrackDonation Service
- Expected Event Type: DONATION_ALLOCATED
- Payload: referenceReqId, details.status, title, description, details.allocatedList
- Action: ระบบจะทำการ Upsert ข้อมูลลงฐานข้อมูล และเปลี่ยนสถานะเป็น DISPATCHED หรือ CANCELLED
- Error Handling: หากไม่มี referenceReqId แนบมา ระบบจะทำการ Log: Skipping event แต่ถ้ามี ID แปลกๆ ส่งมา ระบบจะใช้ลอจิก Upsert สร้างแฟ้มใหม่ให้เพื่อป้องกันข้อมูลสูญหาย

3.2 Producer: บรอดแคสต์สถานะการช่วยเหลือ (Publish)

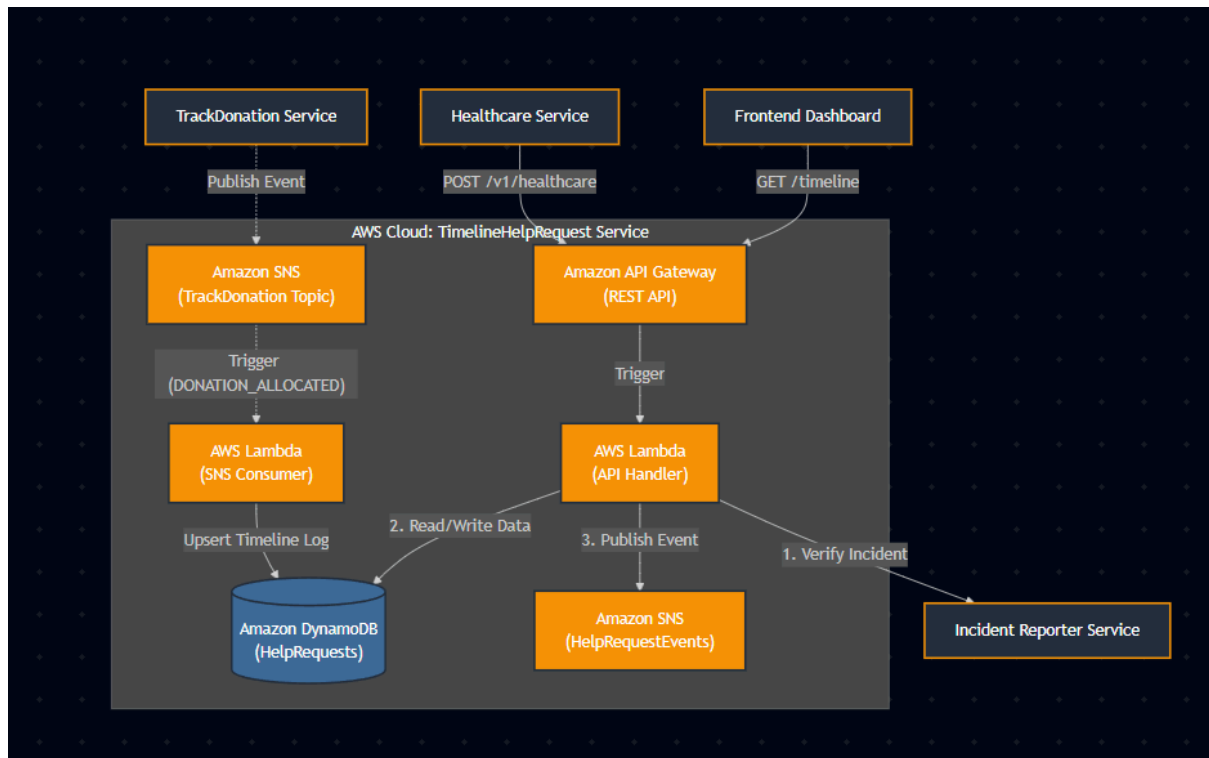
- Interaction Style: Publish/Subscribe
- Channel/Topic: arn:aws:sns:us-east-1:559571211561:HelpRequestEventsTopic
- Event Type: HelpRequestStatusUpdated
- Description: Event จะถูก Publish ทันทีเมื่อมีการสร้างคำร้องใหม่สำเร็จ (ผ่าน POST /v1/help-requests) เพื่อให้ Service อื่นๆ (เช่น Notification Service) รับไปดำเนินการต่อ
- Payload: request_id, incident_id, current_status, timestamp, traceId
- Error Handling: หากการ Publish ไปยัง SNS ล้มเหลว จะถูกดักจับด้วย Try-Catch บล็อกใหญ่ และ Return 500 Internal Server Error

4. Service Data

Database: Amazon DynamoDB (NoSQL) Table Name: HelpRequests โครงสร้างข้อมูล :

- request_id [Partition Key] (String) - รหัสคำร้อง เช่น REQ-ABCD1234
- incident_id (String) - รหัสเหตุการณ์ภัยพิบัติ
- incident_verified (Boolean) - ผลการยืนยันเหตุการณ์กับ Incident Service
- location (Map) - { lat: ..., lon: ... }
- request_type (String) - ประเภทคำร้อง (medical, donation)
- description (String) - รายละเอียด
- current_status (String) - สถานะล่าสุด (NEW, sent_request_hospital, DISPATCHED, CANCELLED ฯลฯ)
- created_at (String - ISO8601 Date) - วันที่สร้าง
- timeline_logs (List of Maps) - ประวัติไทม์ไลน์ ประกอบด้วย:
 - update_id (String) - รหัส UUID ของอัปเดต
 - update_number (Number) - ลำดับการอัปเดต
 - status_at_time (String) - สถานะ ณ เวลานั้น
 - note (String) - ข้อความบันทึก
 - updated_at (String) - เวลาที่อัปเดต
 - updated_by (String) - ผู้ที่ทำการอัปเดต (เช่น Healthcare Service, TrackDonation)

5. Service Architecture



6. Service Interaction

Flow 1: เมื่อมีการแจ้งผู้บาดเจ็บใหม่ (Sync)

1. Healthcare ยิง POST แจ้งข้อมูลผู้บาดเจ็บพร้อม incident_id
2. Timeline ระบุการทำงานชั่วคราวเพื่อยิง GET ไปถาม Incident Service ว่ารหัสนี้มีจริงไหม
3. Timeline บันทึกลง DynamoDB สร้าง request_id ใหม่ และตอบกลับ 200 OK พร้อมรหัสกลับไปให้ Healthcare

Flow 2: เมื่อคลั่งสินค้าเบิกของสำเร็จ (Async)

1. คลั่งสินค้าทำรายการเสร็จสิ้น TrackDonation ยิง Event เข้า SNS Topic ของตนเอง
2. SNS Trigger ปลุก Lambda ของ Timeline ให้ทำงาน (รับข้อมูลแบบ JSON String)
3. Lambda แปลงข้อมูล ค้นหา referenceReqId ใน DynamoDB
4. รวบรวม title, description และ allocatedList จัดฟอร์แมตข้อความ และบันทึกประวัติ (Append Log) สถานะ **DISPATCHED** แบบอัตโนมัติ

7. Dependency Mapping

Upstream Dependencies:

- Incident Reporter Service: จำเป็นสำหรับการ Verify ภัยพิบัติ (มีผลกับฟิลด์ incident_verified แต่ถ้าวระบบนี้ล่ม Timeline ของเรามีระบบ Fallback สามารถทำงานต่อได้)
- TrackDonation Service: เป็นแหล่งข้อมูลเดียวที่จะส่ง Event แจ้งสถานะเบิกจ่ายสิ่งของ (ผ่าน SNS)
- Healthcare Service: เป็นแหล่งข้อมูลหลักในการสร้างเคสผู้ป่วยเจ็บ

Downstream Dependencies:

- Healthcare Service: ต้องรอรับ request_id จากรบบเราในรอบแรก เพื่อนำไปใช้อัปเดตสถานะในรอบต่อไป
- Frontend / Timeline Dashboard: ต้องดึงข้อมูลประวัติผ่าน API ของระบบเรา เพื่อนำไปแสดงผล

8. ลิงค์ที่เกี่ยวข้อง

Github: <https://github.com/Jutinut-BBBOMB/TimeLineHelpRequest>

Video:

<https://drive.google.com/drive/folders/1RVTSKxOq228ELhmT3qpVj6SGnVTtscyl?usp=sharing>

นายจุตินันท์ รัตนมงคลกุล 6609650228